Generating SDKs

Because **FastAPI** is based on the **OpenAPI** specification, its APIs can be described in a standard format that many tools understand.

This makes it easy to generate up-to-date **documentation**, client libraries (**SDKs**) in multiple languages, and **testing** or **automation workflows** that stay in sync with your code.

In this guide, you'll learn how to generate a **TypeScript SDK** for your FastAPI backend.

Open Source SDK Generators

A versatile option is the OpenAPI Generator, which supports **many programming languages** and can generate SDKs from your OpenAPI specification.

For **TypeScript clients**, Hey API is a purpose-built solution, providing an optimized experience for the TypeScript ecosystem.

You can discover more SDK generators on OpenAPI.Tools.

> **Tip**
>
> FastAPI automatically generates **OpenAPI** `3.1` specifications, so any tool you use must support this version.

SDK Generators from FastAPI Sponsors

This section highlights **venture-backed** and **company-supported** solutions from companies that sponsor FastAPI. These products provide **additional features** and **integrations** on top of high-quality generated SDKs.

By ✨ **sponsoring FastAPI** ✨, these companies help ensure the framework and its **ecosystem** remain healthy and **sustainable**.

Their sponsorship also demonstrates a strong commitment to the FastAPI **community** (you), showing that they care not only about offering a **great service** but also about supporting a **robust and thriving framework**, FastAPI. 🙇

For example, you might want to try:

- Speakeasy
- Stainless
- liblab

Some of these solutions may also be open source or offer free tiers, so you can try them without a financial commitment. Other commercial SDK generators are available and can be found online. 🤓

Create a TypeScript SDK

Let's start with a simple FastAPI application:

Ask AI

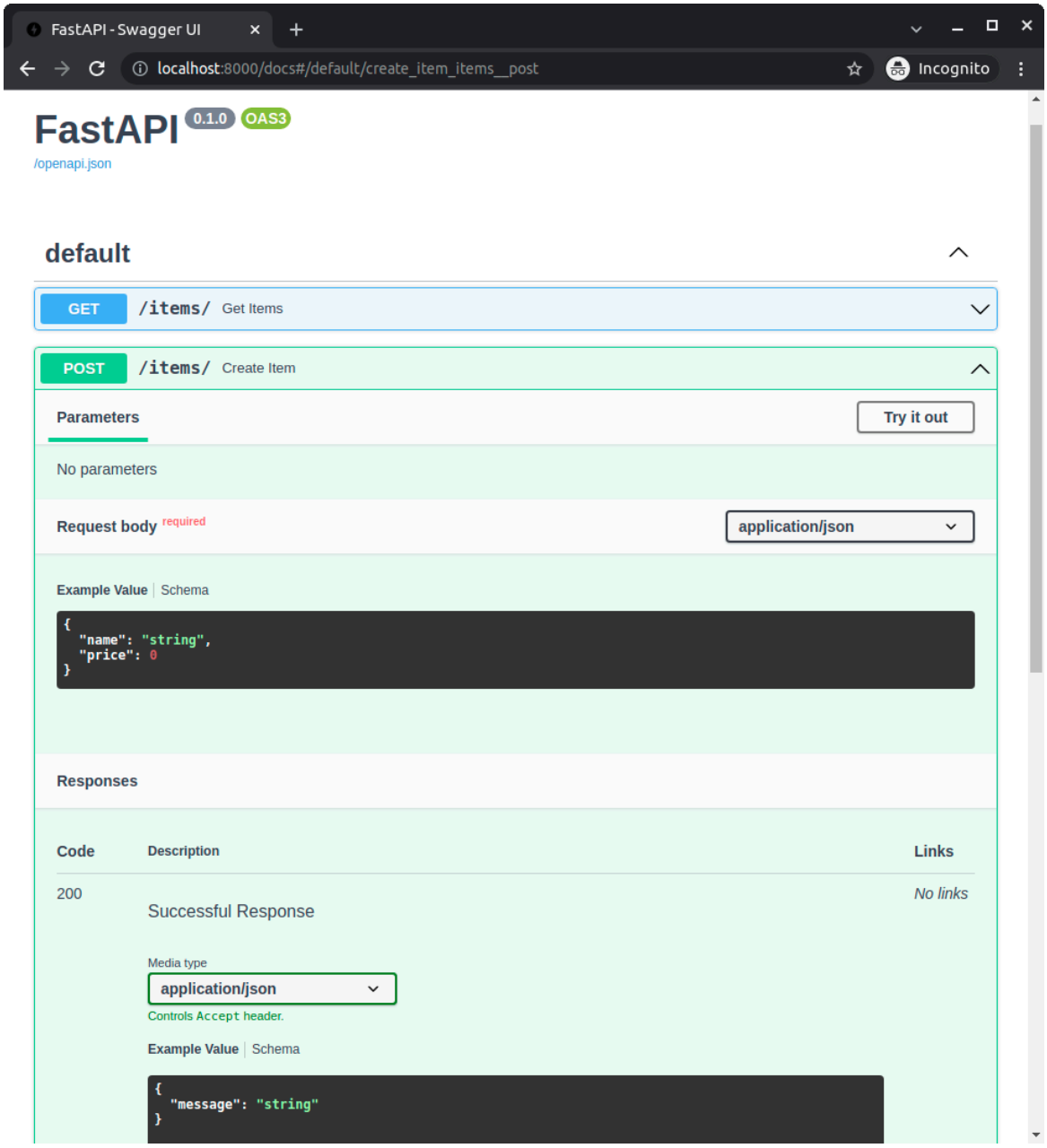
Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    price: float


class ResponseMessage(BaseModel):
    message: str


@app.post("/items/", response_model=ResponseMessage)
async def create_item(item: Item):
    return {"message": "item received"}


@app.get("/items/", response_model=list[Item])
async def get_items():
    return [
        {"name": "Plumbus", "price": 3},
        {"name": "Portal Gun", "price": 9001},
    ]
```

Notice that the *path operations* define the models they use for request payload and response payload, using the models `Item` and `ResponseMessage`.

API Docs

If you go to `/docs`, you will see that it has the **schemas** for the data to be sent in requests and received in responses:



You can see those schemas because they were declared with the models in the app.

That information is available in the app's **OpenAPI schema**, and then shown in the API docs.

That same information from the models that is included in OpenAPI is what can be used to **generate the client code**.

Hey API

Once we have a FastAPI app with the models, we can use Hey API to generate a TypeScript client. The fastest way to do that is via npx.

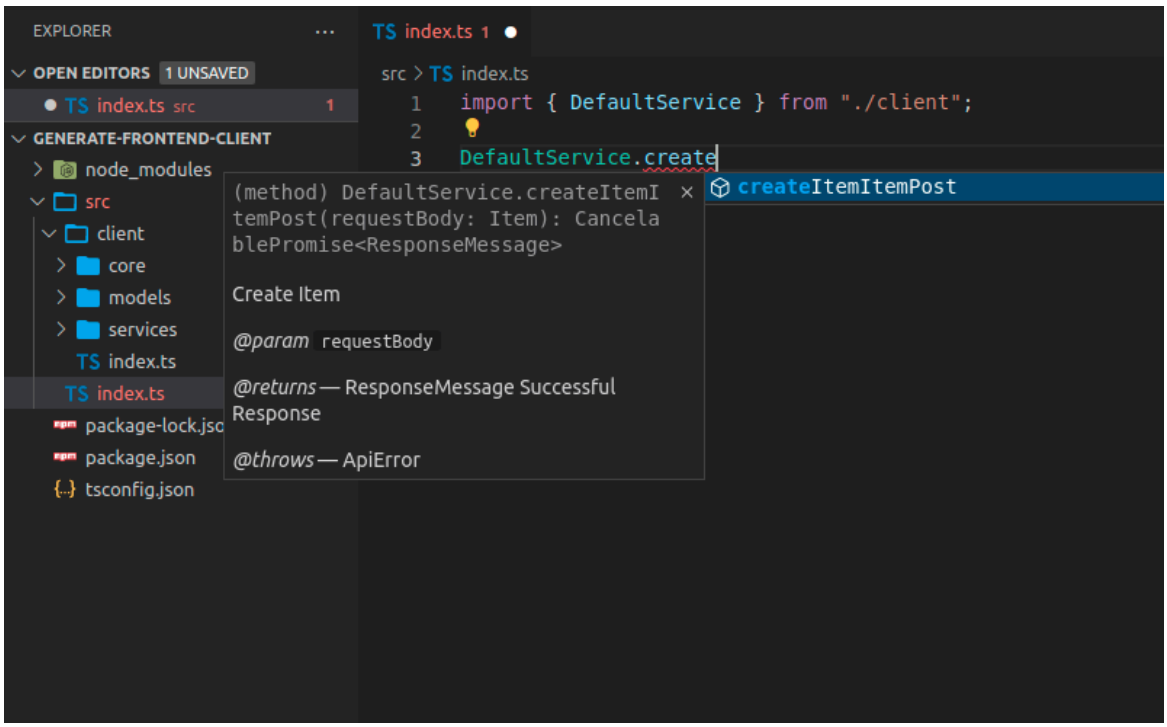
```
npx @hey-api/openapi-ts -i http://localhost:8000/openapi.json -o src/client
```

This will generate a TypeScript SDK in `./src/client`.

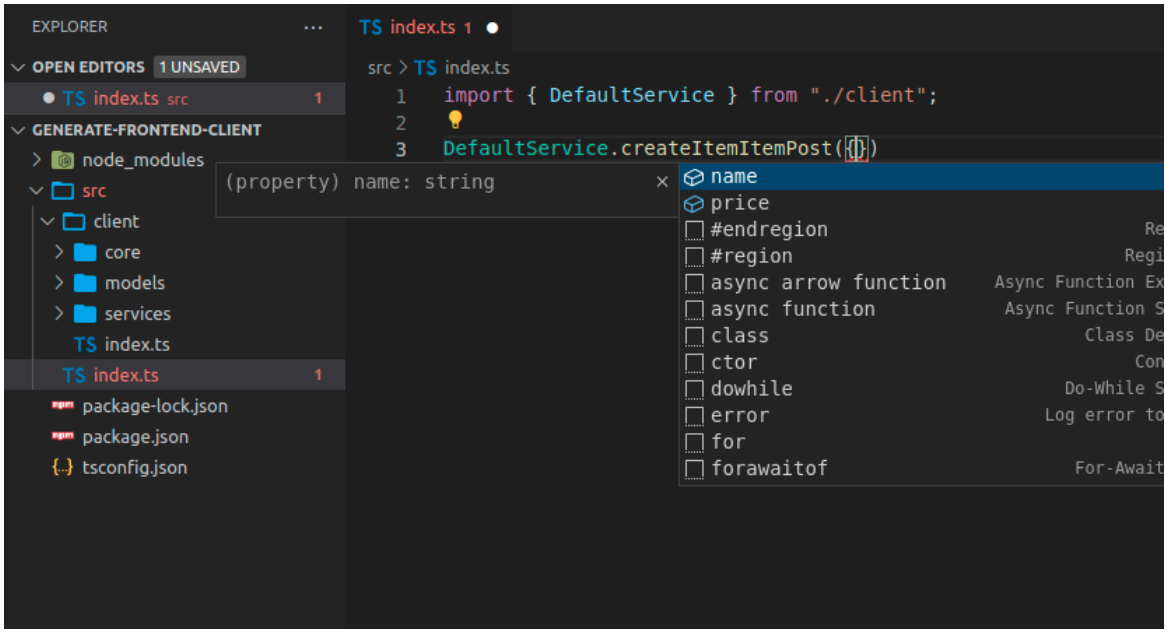
You can learn how to install `@hey-api/openapi-ts` and read about the generated output on their website.

Using the SDK

Now you can import and use the client code. It could look like this, notice that you get autocompletion for the methods:

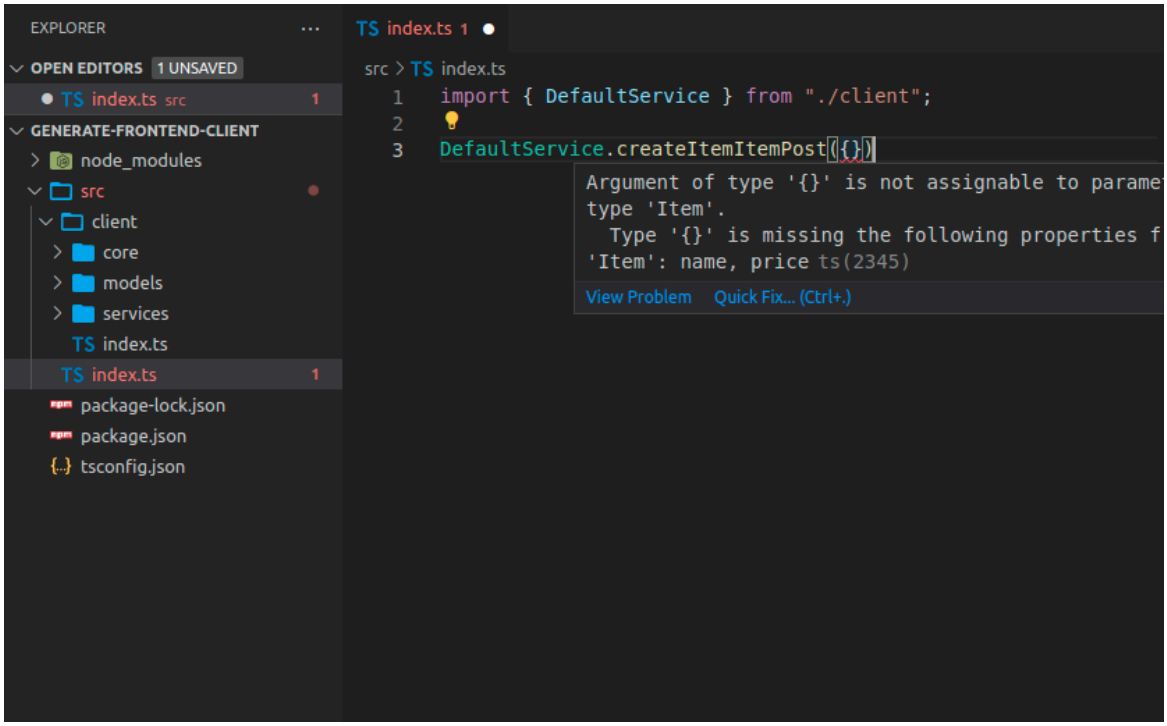


You will also get autocompletion for the payload to send:

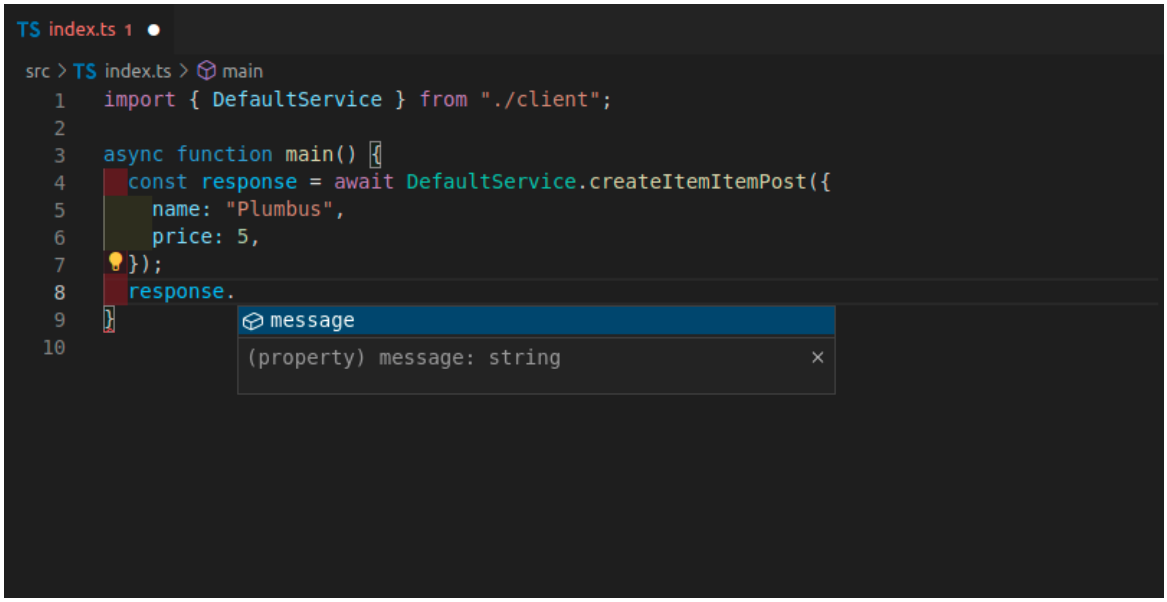


> **Tip**
>
> Notice the autocompletion for `name` and `price`, that was defined in the FastAPI application, in the `Item` model.

You will have inline errors for the data that you send:



The response object will also have autocompletion:



FastAPI App with Tags

In many cases, your FastAPI app will be bigger, and you will probably use tags to separate different groups of *path operations*.

For example, you could have a section for **items** and another section for **users**, and they could be separated by tags:

Python 3.10+

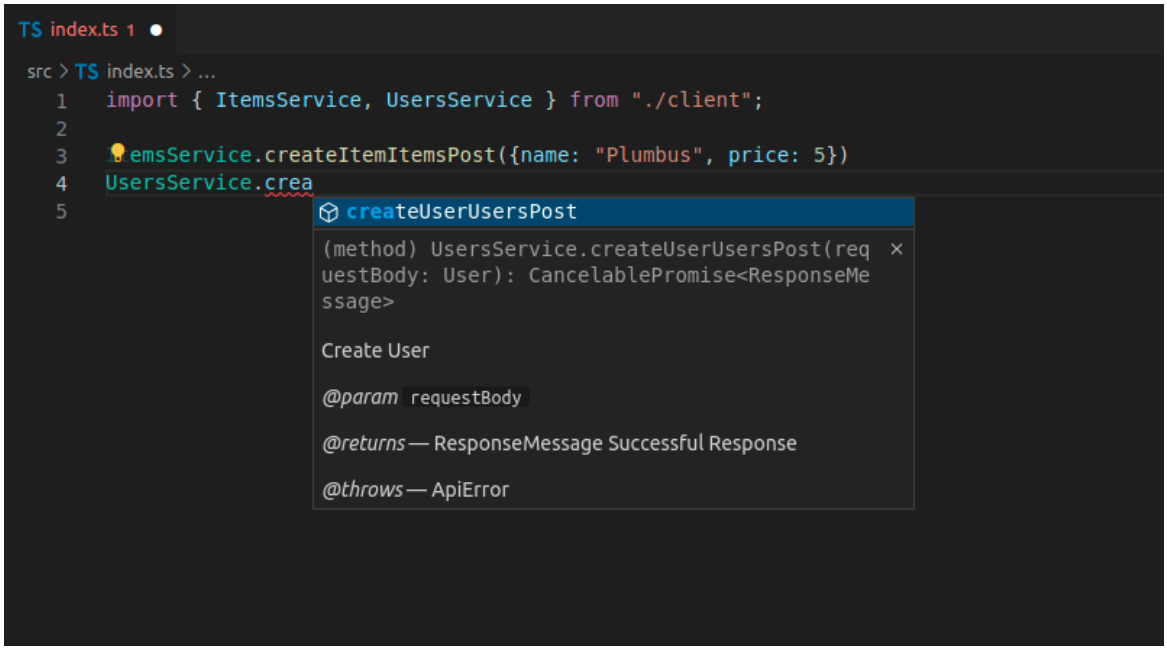
```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
```

```python
class Item(BaseModel):
    name: str
    price: float


class ResponseMessage(BaseModel):
    message: str


class User(BaseModel):
    username: str
    email: str


@app.post("/items/", response_model=ResponseMessage, tags=["items"])
async def create_item(item: Item):
    return {"message": "Item received"}


@app.get("/items/", response_model=list[Item], tags=["items"])
async def get_items():
    return [
        {"name": "Plumbus", "price": 3},
        {"name": "Portal Gun", "price": 9001},
    ]


@app.post("/users/", response_model=ResponseMessage, tags=["users"])
async def create_user(user: User):
    return {"message": "User received"}
```

Generate a TypeScript Client with Tags

If you generate a client for a FastAPI app using tags, it will normally also separate the client code based on the tags.

This way, you will be able to have things ordered and grouped correctly for the client code:



In this case, you have:

- `ItemsService`

- `UsersService`

Client Method Names

Right now, the generated method names like `createItemItemsPost` don't look very clean:

```
ItemsService.createItemItemsPost({name: "Plumbus", price: 5})
```

...that's because the client generator uses the OpenAPI internal **operation ID** for each *path operation*.

OpenAPI requires that each operation ID is unique across all the *path operations*, so FastAPI uses the **function name**, the **path**, and the **HTTP method/operation** to generate that operation ID, because that way it can make sure that the operation IDs are unique.

But I'll show you how to improve that next. 🤓

Custom Operation IDs and Better Method Names

You can **modify** the way these operation IDs are **generated** to make them simpler and have **simpler method names** in the clients.

In this case, you will have to ensure that each operation ID is **unique** in some other way.

For example, you could make sure that each *path operation* has a tag, and then generate the operation ID based on the **tag** and the *path operation* **name** (the function name).

Custom Generate Unique ID Function

FastAPI uses a **unique ID** for each *path operation*, which is used for the **operation ID** and also for the names of any needed custom models, for requests or responses.

You can customize that function. It takes an `APIRoute` and outputs a string.

For example, here it is using the first tag (you will probably have only one tag) and the *path operation* name (the function name).

You can then pass that custom function to **FastAPI** as the `generate_unique_id_function` parameter:

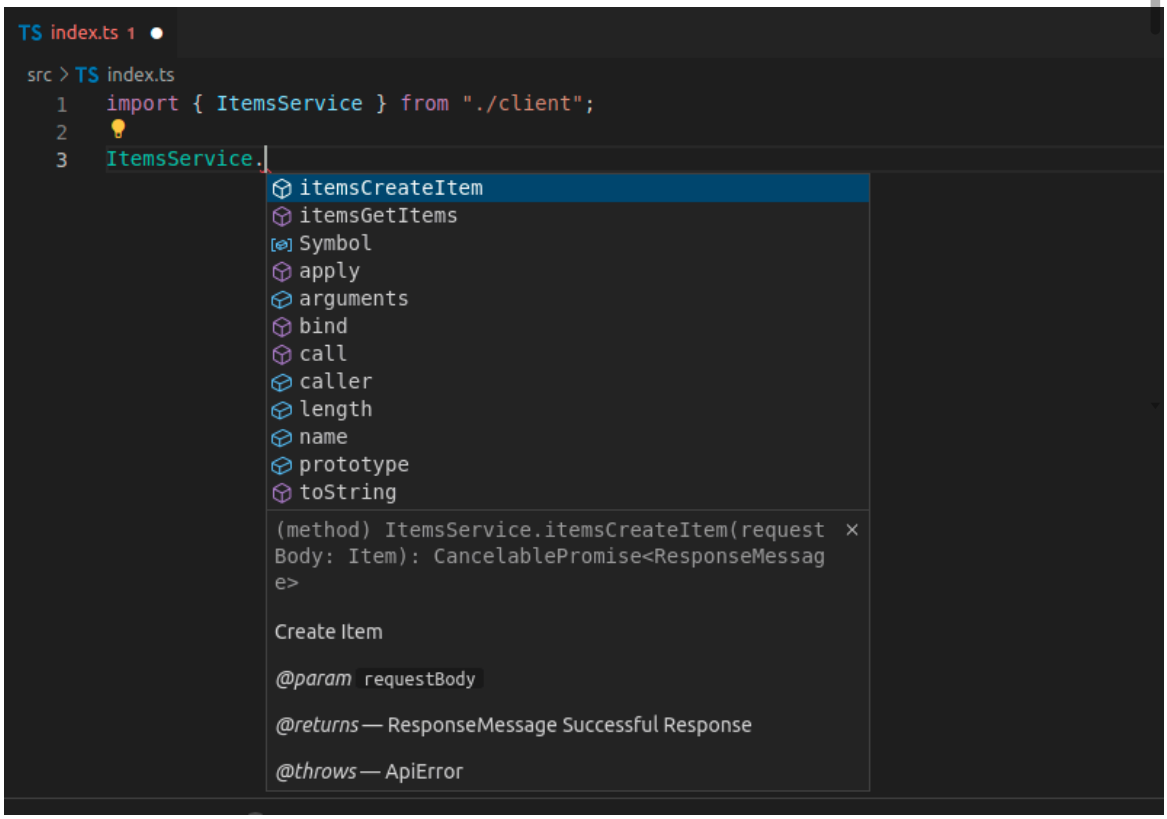
Python 3.10+

```python
from fastapi import FastAPI
from fastapi.routing import APIRoute
from pydantic import BaseModel


def custom_generate_unique_id(route: APIRoute):
    return f"{route.tags[0]}-{route.name}"


app = FastAPI(generate_unique_id_function=custom_generate_unique_id)


class Item(BaseModel):
    name: str
    price: float


class ResponseMessage(BaseModel):
    message: str
```

```python
class User(BaseModel):
    username: str
    email: str


@app.post("/items/", response_model=ResponseMessage, tags=["items"])
async def create_item(item: Item):
    return {"message": "Item received"}


@app.get("/items/", response_model=list[Item], tags=["items"])
async def get_items():
    return [
        {"name": "Plumbus", "price": 3},
        {"name": "Portal Gun", "price": 9001},
    ]


@app.post("/users/", response_model=ResponseMessage, tags=["users"])
async def create_user(user: User):
    return {"message": "User received"}
```

Generate a TypeScript Client with Custom Operation IDs

Now, if you generate the client again, you will see that it has the improved method names:



As you see, the method names now have the tag and then the function name, now they don't include information from the URL path and the HTTP operation.

Preprocess the OpenAPI Specification for the Client Generator

The generated code still has some **duplicated information**.

We already know that this method is related to the **items** because that word is in the `ItemsService` (taken from the tag), but we still have the tag name prefixed in the method name too. 😕

We will probably still want to keep it for OpenAPI in general, as that will ensure that the operation IDs are **unique**.

But for the generated client, we could **modify** the OpenAPI operation IDs right before generating the clients, just to make those method names nicer and **cleaner**.

We could download the OpenAPI JSON to a file `openapi.json` and then we could **remove that prefixed tag** with a script like this:

Python 3.10+

```python
import json
from pathlib import Path

file_path = Path("./openapi.json")
openapi_content = json.loads(file_path.read_text())

for path_data in openapi_content["paths"].values():
    for operation in path_data.values():
        tag = operation["tags"][0]
        operation_id = operation["operationId"]
        to_remove = f"{tag}-"
        new_operation_id = operation_id[len(to_remove) :]
        operation["operationId"] = new_operation_id

file_path.write_text(json.dumps(openapi_content))
```

Node.js

```node.js
import * as fs from 'fs'

async function modifyOpenAPIFile(filePath) {
  try {
    const data = await fs.promises.readFile(filePath)
    const openapiContent = JSON.parse(data)

    const paths = openapiContent.paths
    for (const pathKey of Object.keys(paths)) {
      const pathData = paths[pathKey]
      for (const method of Object.keys(pathData)) {
        const operation = pathData[method]
        if (operation.tags && operation.tags.length > 0) {
          const tag = operation.tags[0]
          const operationId = operation.operationId
          const toRemove = `${tag}-`
          if (operationId.startsWith(toRemove)) {
            const newOperationId = operationId.substring(toRemove.length)
            operation.operationId = newOperationId
          }
        }
      }
    }

    await fs.promises.writeFile(
      filePath,
      JSON.stringify(openapiContent, null, 2),
    )
    console.log('File successfully modified')
  } catch (err) {
    console.error('Error:', err)
  }
}

const filePath = './openapi.json'
modifyOpenAPIFile(filePath)
```

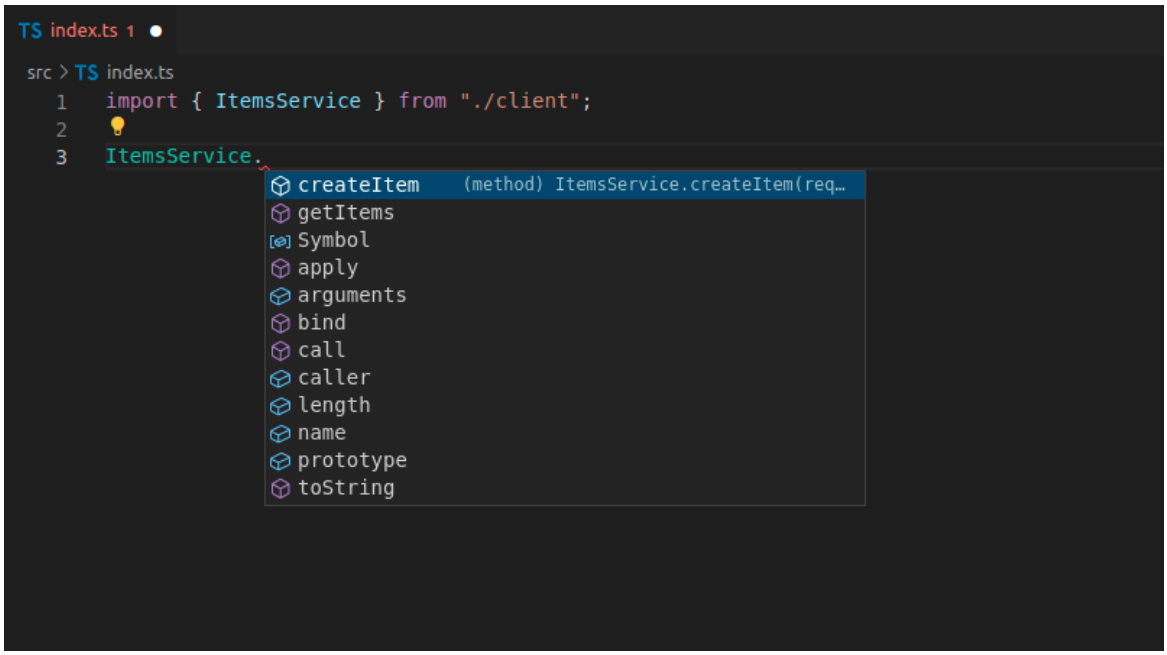
With that, the operation IDs would be renamed from things like `items-get_items` to just `get_items`, that way the client generator can generate simpler method names.

Generate a TypeScript Client with the Preprocessed OpenAPI

Since the end result is now in an `openapi.json` file, you need to update your input location:

```
npx @hey-api/openapi-ts -i ./openapi.json -o src/client
```

After generating the new client, you would now have **clean method names**, with all the **autocompletion**, **inline errors**, etc:



Benefits

When using the automatically generated clients, you would get **autocompletion** for:

- Methods.
- Request payloads in the body, query parameters, etc.
- Response payloads.

You would also have **inline errors** for everything.

And whenever you update the backend code, and **regenerate** the frontend, it would have any new *path operations* available as methods, the old ones removed, and any other change would be reflected on the generated code. 🤓

This also means that if something changed, it will be **reflected** on the client code automatically. And if you **build** the client, it will error out if you have any **mismatch** in the data used.

So, you would **detect many errors** very early in the development cycle instead of having to wait for the errors to show up to your final users in production and then trying to debug where the problem is. ✨